МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
[Назва закладу вищої освіти]
[Назва факультету / інституту]
Кафедра [назва кафедри]

ЗВІТ

з лабораторної роботи № 7
з дисципліни «Глибокі нейронні мережі»

на тему: «Підготовка власного набору зображень в Keras»

Виконав: студент групи 443
Бренькач Д.В.

Перевірила: Дворжак В.В.

[Місто] – 2026

МЕТА, ЗАВДАННЯ ТА ЗАСОБИ

Мета роботи: навчитися завантажувати й використовувати власні набори даних для навчання згорткових нейронних мереж, використовуючи генератори в Keras.

Завдання роботи:

1. Додати автоматичну валідацію до моделі класифікації зображень коней і людей (набір Horses-vs-Humans).
2. Дослідити вплив зменшення розміру зображень з 300x300 до 150x150 на швидкість і якість навчання.
3. Створити генератор і навчити мережу на наборі Happy or Sad до точності 99,9 % менш ніж за 15 епох.
4. Підготувати власний набір даних Dogs vs. Cats, розділити його на навчальний, валідаційний і тестовий, навчити згорткову мережу та перевірити її на власних зображеннях, у тому числі складних і обрізаних.

Програмне забезпечення: Google Colaboratory (графічний процесор NVIDIA T4), Python 3, TensorFlow / Keras 3, Pillow, NumPy, Matplotlib.

Набори даних: Horses-vs-Humans (1 027 навчальних і 256 валідаційних зображень розміром 300x300), Happy or Sad (80 зображень 150x150), Dogs vs. Cats з сайту Kaggle (25 000 зображень з мітками).

Основні поняття. Генератор ImageDataGenerator читає зображення з диска пакетами під час навчання, масштабує значення пікселів до діапазону [0, 1] (параметр rescale), зводить усі зображення до однакового розміру (target_size) і формує правильні відповіді з назв підкаталогів (метод flow_from_directory). Завдяки цьому мережу можна навчати на тисячах зображень, не завантажуючи їх у пам’ять одночасно. Валідаційний набір — дані, які мережа не бачить під час навчання; він показує, як модель працюватиме на нових зображеннях, і дає змогу виявити перенавчання.

ХІД РОБОТИ

Google Colab:
1
2
3
4

1. Завдання 1. Автоматична валідація (Horses-vs-Humans)

До базового ноутбука з лекції додано завантаження валідаційного набору. Навчальний архів horse-or-human.zip розпаковано в каталог ./horse-or-human, валідаційний validation-horse-or-human.zip — у каталог ./validation-horse-or-human. Перевірка вмісту каталогів показала 500 зображень коней і 527 зображень людей у навчальному наборі та по 128 зображень кожного класу у валідаційному.

Модель повторює базовий приклад: вхід 300x300x3, п’ять блоків Conv2D + MaxPooling2D (16, 32, 64, 64 і 64 фільтри), шар Flatten, повнозв’язний шар Dense на 512 нейронів і вихідний нейрон з сигмоїдою (0 — кінь, 1 — людина). Перед шаром Flatten залишається карта ознак 7x7x64, тобто 3 136 входів у шар Dense; загальна кількість параметрів — 1 704 097. Модель скомпільовано з функцією втрат binary_crossentropy, оптимізатором RMSprop (learning_rate = 0,001) і метрикою accuracy.

Крім генератора навчальних даних, створено другий генератор — для валідаційних даних, та передано його в метод fit:

```
train_datagen = ImageDataGenerator(rescale=1/255)
validation_datagen = ImageDataGenerator(rescale=1/255)
train_generator = train_datagen.flow_from_directory('./horse-or-human/',
        target_size=(300, 300), batch_size=128, class_mode='binary')
validation_generator = validation_datagen.flow_from_directory(
        './validation-horse-or-human/',
        target_size=(300, 300), batch_size=32, class_mode='binary')

history = model.fit(train_generator, steps_per_epoch=8, epochs=15, verbose=1,
                    validation_data=validation_generator, validation_steps=8)
```

Генератори знайшли 1 027 навчальних і 256 валідаційних зображень двох класів. Наприкінці кожної епохи, крім loss і accuracy, виводяться val_loss і val_accuracy. Результати навчання наведено в таблиці 1.

Таблиця 1 – Результати навчання моделі 300x300

Показник	**Значення**
Точність на навчальних даних, епоха 15	0,9727
Точність на валідаційних даних, епоха 15	0,7930
Найкраща точність на валідації	0,875 (епоха 10)
Втрати на валідації, епоха 15	1,5594 (мінімум 0,366 на епосі 10)
Час навчання	124,5 с

Після навчання модель перевірено на шести власних зображеннях, знайдених в Інтернеті (табл. 2). Ці самі зображення використано і в завданні 2 (рис. 2).

Таблиця 2 – Розпізнавання власних зображень моделлю 300x300

Зображення	**Очікується**	**Відповідь**	**p_human**	**Правильно**
Темний кінь у засніженому лісі	кінь	кінь	0,499	так (на межі)
Білий кінь на фоні неба й поля	кінь	кінь	0,153	так
Рудий кінь на білому фоні	кінь	кінь	0,000	так
Чоловік у светрі на білому фоні	людина	кінь	0,071	ні
Жінка в капелюсі, яскравий одяг	людина	кінь	0,000	ні
Чоловік у національному вбранні	людина	людина	0,766	так

Висновки до завдання 1

1. Автоматична валідація працює: генератор перевірки бере зображення саме з валідаційного каталогу, а після кожної епохи виводиться точність на даних, яких мережа не бачила.
2. Точність на навчальних даних зросла з 0,55 до 0,97, на валідаційних — з 0,50 до 0,79. Як і очікувалося, на нових даних модель працює гірше. Найкращий стан моделі був на 10-й епосі (0,875), після цього втрати на валідації зростали — ознака перенавчання.
3. Точність на валідації сильно коливалася (від 0,50 до 0,875). Причина — параметр steps_per_epoch = 8 при batch_size = 128: навчальний набір містить 1 027 зображень, тобто 9 пакетів, тому кожна друга епоха отримувала лише залишок у 3 зображення (Keras видав

попередження «Your input ran out of data»). Для стабільного навчання доцільно встановлювати steps_per_epoch = len(train_generator).

4. На власних зображеннях модель правильно розпізнала 4 з 6: усіх трьох коней, але двох людей із трьох прийняла за коней. Навчальні зображення є комп'ютерно згенерованими та мають простий фон, тому мережа значною мірою спирається на колір і фон, а не на форму тіла.

2. Завдання 2. Вплив зменшення розміру зображень

Ноутбук із завдання 1 змінено так: вхідна форма першого згорткового шару — input_shape = (150, 150, 3), у генераторах — target_size = (150, 150), четвертий і п'ятий блоки Conv2D + MaxPooling2D видалено. Підсумок моделі підтвердив, що перший шар видає карту 148x148, а після трьох блоків згортки й об'єднання залишається 17x17x64. Модель навчалася 15 епох з використанням усіх 9 пакетів навчальних даних за епоху.

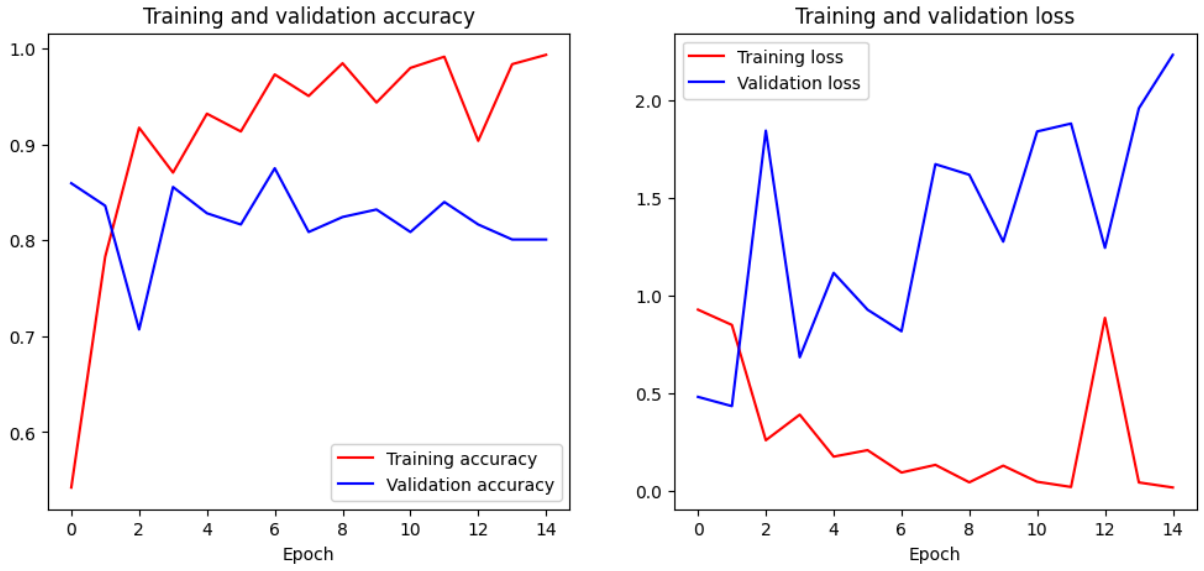


Рисунок 1 – Історія навчання моделі 150x150: точність (зліва) і втрати (справа)

Таблиця 3 – Порівняння моделей із завдань 1 і 2

Показник	**Завдання 1 (300x300, 5 блоків)**	**Завдання 2 (150x150, 3 блоки)**
Карта ознак перед Flatten	7x7x64	17x17x64

Показник	Завдання 1 (300x300, 5 блоків)	Завдання 2 (150x150, 3 блоки)
Входів у шар Dense	3 136	18 496
Усього параметрів	1 704 097	9 494 561
Кроків за епоху	8	9
Час навчання, с	124,5	120,6
Точність на навчанні, епоха 15	0,9727	0,9932
Точність на валідації, епоха 15	0,7930	0,8008
Найкраща точність на валідації	0,875	0,875
Втрати на валідації, епоха 15	1,5594	2,2317
Власних зображень правильно	4 з 6	3 з 6



Рисунок 2 – Розпізнавання власних зображень моделлю 150x150

Модель 150x150 правильно розпізнала білого та рудого коней і чоловіка в светрі, а помилилася на коні в засніженому лісі (p_human = 0,934), жінці в капелюсі та чоловікові в національному вбранні (p_human = 0,000). Тобто відповіді двох моделей на ті самі зображення відрізняються: модель 300x300 правильно розпізнала коня на снігу та чоловіка в національному вбранні, а модель 150x150 — чоловіка в светрі.

Висновки до завдання 2

1. Кількість входів у шар Dense не зменшилася, а зросла — з 3 136 до 18 496. Разом зі зменшенням зображення видалено два блоки згортки, які стискали карту ознак до 7x7. Через це модель має у 5,6 раза більше параметрів, і зменшення зображення пришвидшує лише згорткову частину мережі.
2. Час навчання майже не змінився (120,6 с проти 124,5 с), хоча модель 150x150 обробляла за епоху 9 пакетів замість 8. Точність на валідації обох моделей однакова — близько 0,80, з максимумом 0,875.
3. Модель 150x150 краще запам'ятала навчальні дані (0,993), але має вищі втрати на валідації (2,23 проти 1,56), тобто перенавчилася сильніше. Втрати на валідації зростали майже від 2-ї епохи, а найкращий стан модель мала приблизно на 7-й епосі.
4. На власних зображеннях модель 150x150 впоралася гірше (3 з 6 проти 4 з 6). При зменшенні до 150x150 губляться дрібні деталі (обличчя, ноги, грива), і мережа ще більше спирається на фон, колір і загальний силует.
5. Менші зображення економлять пам'ять і час і підходять для задач, де ознаки класу помітні навіть у малому розмірі. Якщо ж відмінності між класами полягають у деталях, краще використовувати більші зображення. Кількість згорткових шарів треба узгоджувати з розміром входу.
6. Щоб запобігти таким помилкам, набір даних варто комплектувати реальними фотографіями з різним фоном, позами, одягом, освітленням і масштабом, збалансувати класи, а під час навчання застосовувати аугментацію, Dropout і зупинку навчання за втратами на валідації (EarlyStopping).

3. Завдання 3. Набір даних Happy or Sad

Завдання виконано в базовому ноутбуці: код додано лише між рядками START CODE HERE та END CODE HERE. Архів data.zip завантажено в

Colab і розпаковано в каталоги ./data/happy і ./data/sad — по 40 зображень у кожному. Зображення мають розмір 150x150x3, максимальне значення пікселя — 255.

```
train_datagen = ImageDataGenerator(rescale=1./255)
train_generator = train_datagen.flow_from_directory(directory='./data/',
        target_size=(150, 150), batch_size=10, class_mode='binary')

class myCallbackUA(tf.keras.callbacks.Callback):
    def on_epoch_end(self, epoch, logs={}):
        if logs.get('accuracy') is not None and logs.get('accuracy') > 0.999:
            print("\nДосягнуто 99,9% точності, тому скасовуємо навчання!")
            self.model.stop_training = True

model = tf.keras.models.Sequential([
    tf.keras.layers.Input(shape=(150, 150, 3)),
    tf.keras.layers.Conv2D(16, (3, 3), activation='relu'),
tf.keras.layers.MaxPooling2D(2, 2),
    tf.keras.layers.Conv2D(32, (3, 3), activation='relu'),
tf.keras.layers.MaxPooling2D(2, 2),
    tf.keras.layers.Conv2D(64, (3, 3), activation='relu'),
tf.keras.layers.MaxPooling2D(2, 2),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(1, activation='sigmoid')])
model.compile(loss=losses.BinaryCrossentropy(),
              optimizer=optimizers.RMSprop(learning_rate=0.001),
metrics=['accuracy'])
history = model.fit(x=train_generator, epochs=15, callbacks=[callbacks])
```

Генератор знайшов 80 зображень двох класів. Точність на навчальних даних по епохах: 0,39; 0,74; 0,94; 0,83; 0,96; 0,93; 0,99; 0,98; 0,96; 0,95; 1,00. На 11-й епосі точність досягла 1,0000, callback вивів повідомлення «Досягнуто 99,9% точності, тому скасовуємо навчання!» і зупинив навчання. Вимогу (менше 15 епох) виконано.

Перевірочна комірка наприкінці ноутбука виводить повідомлення «Use 'accuracy' as metric», хоча метрику задано правильно. Це особливість Keras 3: властивість model.metrics_names повертає список ['loss', 'compile_metrics'], у якому немає слова accuracy. Те, що callback спрацював за значенням logs['accuracy'], підтверджує правильність метрики.

Висновки до завдання 3

Для простих однорідних зображень (смайли на однаковому фоні) достатньо невеликої мережі з трьома згортковими шарами, щоб за 11 епох досягти 100 % точності на навчальних даних. Callback дає змогу зупинити навчання одразу після досягнення потрібної точності й не витрачати зайві

епохи. Точність зростала нерівномірно, оскільки набір дуже малий (8 кроків по 10 зображень за епоху). Валідаційного набору в цьому завданні немає, тому 100 % показують лише те, що мережа вивчила навчальні дані, а не її якість на нових зображеннях.

4. Завдання 4. Класифікація котів і собак (Dogs vs. Cats)

4.1. Підготовка набору даних

Архів dogs-vs-cats.zip (851,6 МБ) завантажено з сайту Kaggle на Google Drive, скопійовано на локальний диск Colab і розпаковано разом із вкладеними архівами train.zip і test1.zip. Для поділу даних написано власний скрипт prepare_dataset.py. Він:

- визначає клас зображення за назвою файлу (cat.N.jpg, dog.N.jpg) або за назвою батьківської папки;
- пропускає зображення без міток — усі 12 500 файлів з архіву test1.zip, для яких правильні відповіді невідомі;
- відсіює пошкоджені файли за допомогою PIL.Image.verify();
- перемішує зображення кожного класу з фіксованим зерном (seed = 42) і ділить їх у пропорції 70 / 15 / 15 % на навчальний, валідаційний і тестовий набори.

```
def prepare_dataset(raw_dir, out_dir, ratios=(0.70, 0.15, 0.15), seed=42):
    files = [p for p in Path(raw_dir).rglob('*') if p.suffix.lower() in EXTS]
    by_class = {'cats': [], 'dogs': []}
    for p in files:
        lab = label_of(p)
        if lab is not None and is_valid_image(p):
            by_class[lab].append(p)
    rnd = random.Random(seed)
    for cls, items in by_class.items():
        items = sorted(items); rnd.shuffle(items)
        n = len(items); n_train = int(n * ratios[0]); n_val = int(n * ratios[1])
        parts = {'train': items[:n_train],
                 'validation': items[n_train:n_train + n_val],
                 'test': items[n_train + n_val:]}
        for split, lst in parts.items():
            dst = Path(out_dir) / split / cls
            dst.mkdir(parents=True, exist_ok=True)
            for i, src in enumerate(lst):
                shutil.copy(src, dst / f"{cls[:-1]}.{i}{src.suffix.lower()}")
```

Скрипт знайшов 37 500 зображень: 12 500 котів, 12 500 собак і 12 500 без міток; пошкоджених файлів не виявлено. Отриману структуру каталогів наведено в таблиці 4.

Таблиця 4 – Структура каталогів підготовленого набору

Каталог	cats	dogs	Разом
dataset/train/	8 750	8 750	17 500
dataset/validation/	1 875	1 875	3 750
dataset/test/	1 875	1 875	3 750

4.2. Генератори та модель

Для кожного з трьох наборів створено генератор ImageDataGenerator(rescale = 1/255) з параметрами target_size = (150, 150), batch_size = 32, class_mode = 'binary'; для тестового генератора вимкнено перемішування (shuffle = False). Мітки беруться з назв каталогів: cats = 0, dogs = 1. Генератори знайшли 17 500, 3 750 і 3 750 зображень відповідно.

Використано архітектуру мережі з лекції: три блоки Conv2D + MaxPooling2D (16, 32, 64 фільтри), шар Flatten (18 496 значень), Dense на 512 нейронів і вихідний нейрон із сигмоїдою, що видає ймовірність класу «собака». Загальна кількість параметрів — 9 494 561. Мережу навчено двічі, по 15 епох, з оптимізаторами Adam і RMSprop (learning_rate = 0,001), після чого кожну модель оцінено на тестовому наборі.

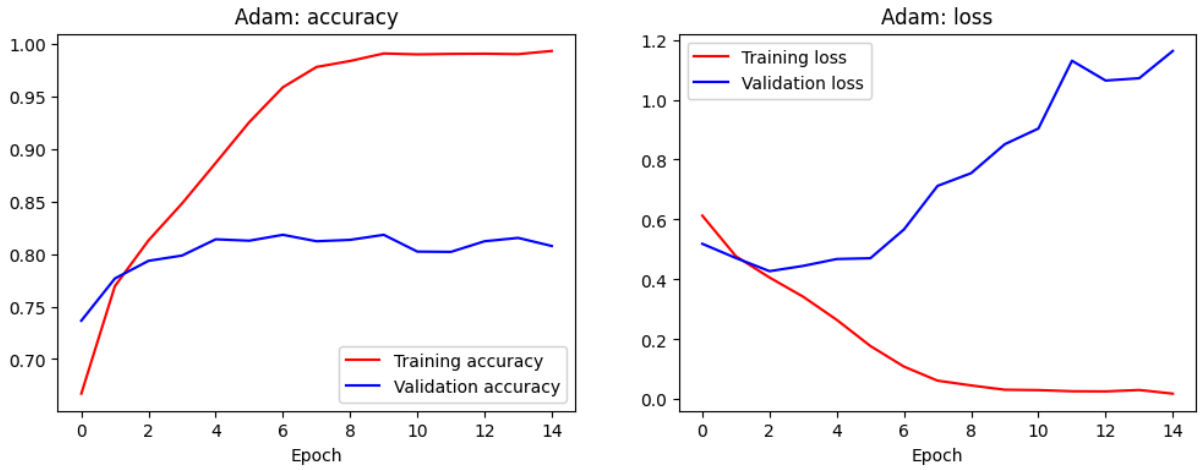


Рисунок 3 – Історія навчання з оптимізатором Adam

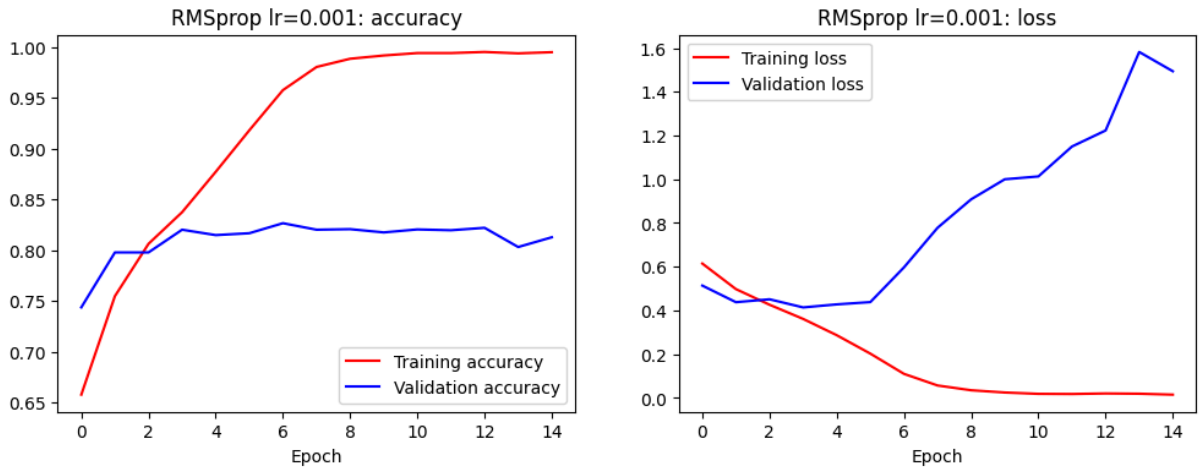


Рисунок 4 – Історія навчання з оптимізатором RMSprop (learning_rate = 0,001)

Таблиця 5 – Порівняння оптимізаторів і оцінка на тестовому наборі

Оптимізатор	**Точність навч.**	**Точність вал.**	**Точність тест**	**Мін. втрати вал. (епоха)**	**Втрати вал., епоха 15**	**Час, с**
Adam	0,9936	0,8077	0,8027	0,428 (3)	1,162	651
RMSprop, lr = 0,001	0,9952	0,8128	0,8205	0,414 (4)	1,494	625

Обидві моделі перенавчаються: після 4–5-ї епохи точність на навчальних даних наближається до 0,99, точність на валідації залишається на рівні 0,80–0,83, а втрати на валідації зростають у 3–3,5 раза. Точність на тестовому наборі майже збігається з валідаційною, отже поділ даних коректний і витоку між наборами немає. RMSprop дав трохи кращий результат (+0,5 % на валідації, +1,8 % на тесті), але ця різниця в межах випадкового розкиду одного запуску, а характер навчання не змінився. Для подальшої роботи обрано модель з RMSprop (за точністю на валідації); її збережено у файл cats_dogs_cnn.keras (72 МБ) на Google Drive, щоб не перенавчати мережу повторно.

4.3. Розпізнавання власних зображень

Модель перевірено на 19 власних зображеннях (11 котів і 8 собак), серед яких були й складні: незвичні пози, одяг і аксесуари, різний фон, кілька

тварин у кадрі (рис. 5, табл. 6). Перед розпізнаванням кожне зображення зведено до розміру 150x150 і поділено на 255 — так само, як у генераторах.



Рисунок 5 – Результати розпізнавання власних зображень (p_dog — ймовірність класу «собака»)

Таблиця 6 – Результати розпізнавання власних зображень

Файл	**Опис**	**Відповідь**	**p_dog**	**Правильно**
cat	Сірий британський кіт крупним планом	кіт	0,000	так
cat1	Рудий кіт у стрибку на білому фоні	собака	0,901	ні
cat2	Коричневий кіт сидить, білий фон	кіт	0,000	так
cat3	Чорний кіт на природному фоні	кіт	0,000	так
cat4	Пухнастий кіт звісив лапи з краю	кіт	0,000	так
cat5	Кіт лежить у раковині	кіт	0,000	так
cat6	Кіт у рожевих окулярах	кіт	0,000	так
cat7	Кіт в окулярах, кофті й з ланцюгом	собака	1,000	ні
cat8	Смугастий кіт сидить як людина	кіт	0,000	так
cat9	Рудий пухнастий кіт, темний фон	кіт	0,000	так
cat10	Смугастий кіт з великими очима	кіт	0,000	так
dog	Бігль на гравії	собака	1,000	так
dog1	Німецька вівчарка на траві	собака	0,957	так
dog2	Чорний дог, розмитий строкатий фон	собака	0,660	так
dog3	Акіта на білому фоні	собака	1,000	так
dog4	Хаскі на снігу	кіт	0,379	ні
dog5	Китайська чубата на траві	собака	0,997	так
dog6	Два сенбернари на траві	собака	1,000	так
dog7	Доберман на сухій траві	собака	0,777	так

Правильно розпізнано 16 зображень з 19 (84 %), що відповідає точності на тестовому наборі. Помилки: кіт cat1 у незвичній позі (витягнуте тіло в стрибку), кіт cat7, якого майже повністю закривають одяг і аксесуари, та хаскі dog4 зі світлою пухнастою шерстю на однорідному сніговому фоні. Найменша впевненість (p_dog = 0,66 і 0,78) — на темних собаках на строкатому фоні трави й листя.

4.4. Складні зображення

Додатково перевірено нові зображення, де тварина невелика відносно кадру, а фон строкатий (трава, дерева, людина поруч): cat11, cat12, dog11–dog15. Разом із попередніми модель помилилася на трьох зображеннях (табл. 7).

Таблиця 7 – Неправильно розпізнані складні зображення

Файл	**Насправді**	**Відповідь моделі**	**p_dog**
cat6	кіт	собака	0,993
cat11	кіт	собака	1,000
dog15	собака	кіт	0,375

Більшість складних зображень модель розпізнала правильно, тому вдалося знайти три помилки замість запланованих 5–6. Зображення cat6 у першій перевірці було розпізнано правильно (p_dog = 0,000), а в цьому запуску — неправильно: на межових зображеннях відповідь нестійка й залежить від конкретних значень ваг навченої моделі.

4.5. Обрізка неправильно розпізнаних зображень

Зображення, на яких модель помилилася або була невпевнена, обрізано до центральної частини (70 % ширини та 80 % висоти) і розпізнано повторно. Результати наведено на рис. 6 і в таблиці 8.



Рисунок 6 – Зображення до і після обрізки

Таблиця 8 – Результати розпізнавання до і після обрізки

Файл	**Насправді**	**До обрізки**	**Після обрізки**	**Результат**
dog4	собака	кіт (0,379)	собака (0,996)	виправлено
dog2	собака	собака (0,660)	собака (1,000)	впевненість зросла
cat1	кіт	собака (0,901)	собака (0,718)	без змін
cat7	кіт	собака (1,000)	собака (0,999)	без змін
cat5	кіт	кіт (0,000)	собака (0,912)	погіршилось
dog7	собака	собака (0,777)	кіт (0,000)	погіршилось

Висновки до завдання 4

1. Власний скрипт підготовки даних коректно розділив набір Dogs vs. Cats на навчальний (17 500), валідаційний (3 750) і тестовий (3 750) набори зі збалансованими класами, а генератори ImageDataGenerator дали змогу навчати мережу на 17 500 зображеннях без завантаження їх у пам’ять.
2. Згорткова мережа з лекції досягла точності близько 82 % на тестовому наборі та 84 % на власних зображеннях. Водночас вона сильно перенавчається: точність на навчальних даних — 99 %, а найкращою модель була вже на 4–5-й епосі.

3. Заміна оптимізатора Adam на RMSprop з learning_rate = 0,001 суттєво не покращила якість навчання: різниця в точності становить 0,5–1,8 % і не виходить за межі випадкового розкиду.
4. Коли тварина займає малу частину кадру, після стиснення до 150x150 від неї залишається кілька десятків пікселів, і мережа ухвалює рішення здебільшого за фоном і контекстом. У навчальних даних фон пов’язаний з класом: собаки частіше сфотографовані на вулиці, коти — вдома.
5. Обрізка допомагає, коли прибирає фон і залишає тварину цілою (хаскі: «кіт» 0,379 → «собака» 0,996). Якщо обрізка відтинає голову, результат погіршується (доберман: «собака» 0,777 → «кіт» 0,000), бо саме морда й голова є головною ознакою для розрізнення котів і собак. Якщо помилку спричиняє сама тварина (кіт в одязі), обрізка не допомагає.
6. Покращити якість можна за допомогою аугментації даних (повороти, зсуви, масштабування, віддзеркалення), шарів Dropout, зупинки навчання за втратами на валідації (EarlyStopping), глибшої мережі або перенавчання попередньо навчених моделей (transfer learning: MobileNetV2, VGG16). На практиці доцільно спочатку знаходити тварину на зображенні детектором об’єктів, а класифікувати вже обрізаний фрагмент.

ЗАГАЛЬНІ ВИСНОВКИ

У лабораторній роботі освоєно повний цикл роботи з власним набором зображень у Keras: завантаження архіву, поділ даних на навчальний, валідаційний і тестовий набори власним скриптом, подача даних у модель через генератори ImageDataGenerator, навчання з валідацією, оцінка на тестовому наборі та перевірка на власних зображеннях.

1. Генератори ImageDataGenerator з методом flow_from_directory зручно й ефективно готують дані: читають зображення з диска

пакетами, нормалізують пікселі, зводять зображення до одного розміру й формують мітки з назв каталогів.

2. Валідаційний набір є обов’язковим. Без нього точність 97–100 % на навчальних даних (завдання 1–4) сприймалася б як успіх, хоча на нових даних моделі дають лише 79–82 %.
3. Основна проблема простих згорткових мереж на цих наборах — перенавчання: після 4–10 епох втрати на валідації починають зростати. Для боротьби з ним використовують EarlyStopping, аугментацію і Dropout.
4. Розмір вхідних зображень і глибина мережі мають бути узгоджені: зменшення входу до 150x150 без зміни решти архітектури збільшило кількість параметрів у 5,6 раза і не пришвидшило навчання.
5. На реальних фотографіях мережі спираються не лише на сам об’єкт, а й на фон і контекст, тому помиляються на незвичних позах, одязі, снігу та строкатому фоні. Правильна обрізка, що залишає об’єкт цілим у центрі, виправляє частину помилок, а невдала — створює нові.
6. Для практичного використання навчальний набір слід розширювати й урізноманітнювати, а також застосовувати перенавчання попередньо навчених мереж.